

E-commerce User Behavior Analysis & Recommendation System

Kyle Naluan

School of Science and Engineering
University of Missouri Kansas-City
Kansas City, Missouri, United States

Hallee Pham

School of Science and Engineering
University of Missouri Kansas-City
Kansas City, Missouri, United States

Kai White

School of Science and Engineering
University of Missouri Kansas-City
Kansas City, Missouri, United States

Morgan Sansone

School of Science and Engineering
University of Missouri Kansas-City
Kansas City, Missouri, United States

Abstract—E-commerce platforms track behavior signals—clicks, scrolls, page views, search queries, and session duration. Most retailers don't show the actual insight into this information. This paper presents a seven-component data science pipeline under a publicly available multi-category e-commerce dataset showing millions of events across two months of activity. The pipeline goes from exploratory analysis of the user patterns, category-based collaborative filtering, behavior segmentation, an A/B testing framework, a time-sensitive recommended system, and a purchase timing predictor based on historical price fluctuations.

Keywords—collaborative filtering, user segmentation, A/B testing, customer journey analysis, price predictor, e-commerce analytics.

I. INTRODUCTION

A. Background

The e-commerce industry has experienced exponential growth since the widespread adoption of the internet. Its reliability and convenience have provided consumers with the ability to purchase retail products from all around the world. The e-tailer can track user behavior data and retailer patterns to give insight to businesses and buyers alike.

B. Project Aims

This project aims to analyze user browsing behavior and purchasing patterns across a large-scale e-commerce dataset [1], then use those insights to build on several other tasks. A user-based collaborative filtering recommendation system is built to show relevant products for each shopper, with user segmentation based on behavior that groups users into clusters by interaction history. An A/B testing analysis on different user groups evaluates the real-world impact of personalized recommendations against a standard baseline across user groups. A time-sensitive recommendation system feature that considers trends, time, and dates. Finally, a customer-product history system that visualizes customer journey from view to purchase or abandonment, with an integrated price-timing predictor suggesting optimal time for purchases.

II. RELATED WORK

A. Recommendation System

Collaborative filtering (CF) is a dominant approach for making personalized recommendations in e-commerce. It is based on the principle that users with similar interaction

histories will prefer similar items. CF methods fall into two categories: user-based (find similar users and recommend what they liked), and item-based (find similar items based on shared user interactions). These two approaches formed the starting point for the recommendation component of this project.

Linden et al. demonstrated at Amazon that item-based CF outperforms both user-based CF and cluster models at scale. They found that item-item relationships are more stable over time than user-user similarities and also scale better computationally [2]. This finding justified the choice of category-level item-based CF in this project.

For similarity measurement, Hu et al. identified unique properties of implicit feedback datasets (clicks, views, and purchases). They showed that they must be treated differently from explicit ratings, since implicit data reflects behavior rather than stated preference. It also does not naturally capture negative feedback [3]. Cosine similarity is commonly used in this setting because it focuses on the pattern of interactions across all items rather than overall user activity volume. This makes it well-suited for behavior data where interaction frequency varies widely across users. This informed the choice of cosine similarity as the similarity measure for this project's CF implementation.

Koren from Yahoo! Research identified that temporal dynamics are a weakness of standard CF. He showed that user preferences and item popularity shift significantly over time and that models treating all interactions equally, regardless of when they occurred, fail to capture how preferences evolve over time [4]. This finding encouraged time-aware recommendations, where things like day and seasonal patterns are used to adjust recommendation scores without fully rebuilding the underlying model. This approach inspired this project's time-sensitive recommendation component.

B. Customer Segmentation

Customer segmentation is the process of dividing a user base into groups based on shared behavioral characteristics. This allows for more targeted marketing, personalization, and product strategy. Alves Gomes and Meisen identified a commonly used four-step customer targeting process: data collection, customer representation via feature selection, customer analysis via segmentation, and customer targeting [10]. This structure directly reflects our work pipeline.

RFM analysis is identified as the most popular feature selection method, appearing in 41.9% of reviewed

publications, valued for its simplicity, scalability, and interpretability [10]. Our `user_features.parquet` follows this approach, and also extends standard RFM metrics with additional behavioral features, which is another common tactic.

K-means is the dominant segmentation method across the reviewed literature (39% of publications), attributed to its scalability and compatibility with RFM-derived features [10]. The review notes that no consensus exists on evaluation metrics due to the absence of ground truth, which motivated our use of both the elbow method and silhouette score together to select the number of clusters.

C. Price Prediction

The problem of predicting optimal purchase timing was written by Etzioni et al. [5], who looked into historical airfare data to minimize ticket purchase price to determine whether the user should buy now or wait for a lower price. Their rule-based system shows that price patterns are always moving, which can push the work for using this “buy now or wait” method for users in many other contexts. Specifically in e-commerce, price fluctuations are driven by inventory levels, promotional calendars, and competitors. These become irregular and even more irregular for air fares, but can be helped by data-driven predictors. Purchase-timing prediction in e-commerce has always been framed as a regression task. Recent work has shown that, instead of regressing this price prediction to an exact future price, reframing it as a binary classification produces more actionable outputs and better estimates.

Gradient-boosting decision trees became a large model for e-commerce predictors. Song and Liu [6] applied XGBoost to pre-cut purchase behavior on an e-commerce platform, this show regularize boosting procedure outperforms in terms of precision, recall, and F1-score under cross-validation, while Liu et al. [7] demonstrated that an XGBoost-logistic regression model further improves the repurchase predictors by combining the nonlinear expressiveness of tree ensembles with the stability of linear. Both studies show that gradient boosting classifiers can handle the sparse, large datasets on online retail data. Our purchase predictor uses scikit-learn’s HistGradientBoostingClassifier [8]. The classifier uses rolling price statistics: 7-day mean, standard deviation, and a price to 30-day minimum ratio, to show when the predicted price is to fall below its historical reference of a previous price data. This is consistent with the classification paradigm shown to be effective across other e-commerce behavior prediction tasks [6, 7].

III. METHODOLOGY

A. Data Acquisition and Exploratory Data Analysis

Data was sourced from the Kaggle “eCommerce Behavior Data from Multi-Category Store” data set [1], consisting of two CSV files covering October and November 2019 with a combined 109,950,743 rows across 9 columns. Exploration was conducted in two phases. The first phase loaded a 100,000-row sample from the October file to quickly assess the schema, column data types, missing value patterns, and surface-level data quality issues. The second phase performed a chunk-based pass over both complete files in chunks of 100,000 rows at a time, calculating statistics across all 110 million rows to produce accurate dataset-wide counts. This

chunking approach was necessary due to the file sizes (5.6 GB and 9 GB), which exceeded available memory for a single load. Each chunk was checked for missing values, duplicate rows, zero price rows, null session IDs, event type distribution, unique user and product counts, and cross-month user overlap.

B. Preprocessing

The initial preprocessing addressed the following data quality issues: missing category/brand values, duplicate observations, prices logged as zero, and incorrect datatypes. Additionally, datatypes were downcast to further optimize storage.

Similar to the initial dataset exploration, the two raw CSV files were handled with chunking. Each chunk of raw data was loaded and cleaned individually. Clean chunks were combined into a single clean dataset for each respective month. A chunk size of 500,000 rows was chosen to maximize throughput without exceeding the 30 GB memory limit provided by Kaggle notebooks.

The final clean data was saved as a parquet file due to better compression, storage efficiency, and read speeds. The fastparquet engine was used to allow appending the two cleaned dataframes for October and November into a single output file, `ecommerce_oct_nov_clean.parquet`.

The purpose of this notebook was simply to clean and handle data quality issues.

C. Feature Engineering

Feature engineering was centralized in a single notebook. The only input was the cleaned dataset from preprocessing, which was used to produce/output four feature tables to be used in downstream notebooks. These outputs extracted essential behavioral relationships from the base data. They serve as a basis for later modeling/analysis. They can be used as-is or operated upon further. The production of these feature tables enforces feature consistency amongst models and prevents redundant reprocessing of the base event logs by aggregating features only once. The feature outputs captured user, session, and product behavior.

The first output, `user_feature.parquet`, aggregated event logs into a single table with one row per user. Primarily designed for use in customer segmentation, this table captures RFM metrics per user. It also captures additional behavioral metrics such as engagement, category/brand preferences, and various rates.

The `session_features.parquet` extracted information about individual sessions. These metrics included volume of events, time (date/duration/etc), session outcome flags, and revenue. The goal of this output was to aid in visualizing customer history/journeys and making time-based recommendations.

The third output, `user_category_affinity.parquet`, captured the strength of relationships between individual users and each category. For each user-category pair, a weighted affinity score was assigned based on the number of events ($views \times 1 + carts \times 3 + purchases \times 5$). Weights were assigned to events based on potential/ability to result in revenue. This feature table is the basis for our recommendation systems.

The final output, `product_price_history.parquet`, tracks the daily median price for each product in the dataset. Additionally, 30-day rolling statistics and price percentiles were calculated for each product $\times$ date. This output is the primary input for our price timing predictor.

In order to optimize memory, the clean dataset was loaded separately for each output with only the columns necessary to derive those features. Loading the entire dataset and creating new features exceeded our allocated memory limits. After saving and analyzing each output, the clean dataset and derived table were deleted, and the Python Garbage Collector was used to ensure that memory was freed.

D. User Segmentation

User segmentation was performed using the user_feature data produced in feature engineering. Nine numeric features were chosen for clustering, including RFM/engagement metrics and conversion/abandonment rates. K-means clustering, which utilizes Euclidean distance and takes numeric inputs, was chosen as the algorithm to separate users based on these features.

Prior to clustering, the data had to be normalized and standardized to ensure fair distance calculations. Right-skewed features were handled with a logp1 transform, and left-skewed features were cubed. After normalization, scikit-learn's StandardScaler was used on the data to ensure that the magnitude of feature measurements wouldn't affect cluster assignment.

Clustering was performed on a sample of 500,000 users for different k numbers of clusters, ranging from 2 to 10. The final k number of clusters was chosen by evaluating clustering performance using the elbow method and silhouette score. K-means clustering was fit over the entire user base using the selected k. The final clusters were given subjective labels based on median statistics per cluster.

E. Collaborative Filtering

The collaborative filtering component was built using the user-category affinity matrix produced in the feature engineering phase. A minimum affinity threshold of 5 was applied to filter interactions unlikely to represent genuine interest, given how views, cart additions, and purchases are weighted. The filtered matrix consisted of 2,227,793 users across 13 top-level categories.

CF was implemented at the category level rather than the product level because product-level interactions would be extremely sparse across 190,000 products, making similarity computation unreliable. User-based CF was considered but rejected because a user-user similarity matrix of 2.2×2.2 million users was computationally infeasible, and 74.54% of users interacted with only one category after filtering, leaving insufficient cross-category signal per user. Category-level item-based CF was chosen instead.

Cosine similarity was computed on the resulting 13×13 category-category similarity matrix. Cosine similarity was chosen over Pearson correlation because it is designed for implicit behavioral data [3]. After comparing factors of 25, 50, and 100 side by side, a shrinkage factor of 50 was chosen because it was aggressive enough to reduce category pairs with fewer than 10 shared users to near-zero, while conservative enough to avoid over-penalizing mid-range pairs.

For each user, unseen categories were ranked by predicted affinity score. Evaluation used a leave-one-out split, holding out each user's highest affinity category as the test item, since failing to predict a user's strongest category preference would be a meaningful failure signal. A temporal split would have been used, but it was not feasible because the affinity scores

were aggregated across both months. Users with only one category were excluded from evaluation but kept in the interaction matrix since they still contribute a signal to the similarity computation. K was set to 5 to ensure enough recommendation slots relative to the 12 unseen categories available to most users. Recall@5 was used as the evaluation metric, with a random baseline of 43.45% serving as the lower bound of comparison. This was computed empirically rather than using the theoretical estimate of 41.7% (5/12), since users have varying numbers of unseen categories after filtering.

F. Time-Sensitive Recommendations

The time-sensitive recommendation component extends the CF baseline as a post-processing re-ranking layer, without rebuilding the underlying model. Category-level conversion lift factors were computed per hour of day and day of week. For the seasonal dimension, daily lift factors were computed at the overall level using lift = conversion rate on date / overall conversion rate.

The seasonal layer was determined by letting the data define significance rather than using a fixed date window. November 15 was excluded due to evidence of a data collection failure. November 17 was kept as a high-converting promotional event. Each user's most recent session time was used as their temporal context, as it is data-driven and generalizable across all users.

The three lift factors were combined by multiplication, consistent with standard lift-based scoring in marketing analytics. Shrinkage was applied to smooth lift factors for sparse categories (using the same formula as the CF similarity matrix). A factor of 5,000 was chosen based on the natural gap in purchase counts between sport (7,722) and medicine (1,960) categories.

Recall@5 and NDCG@5 were used as evaluation metrics. Recall@5 alone is insufficient because re-ranking changes position within the top 5, but not which categories are included. NDCG@5 is position-sensitive, rewarding correct categories ranked higher.

G. A/B Testing

A/B Testing was conducted on the user segments to confirm key metrics would be different between them. The key metrics were the true purchase rate, true cart rate, and the distributions of view count, time spent browsing, and money spent. These metrics were analyzed because of their importance to the overall buying experience. Higher purchase and cart rates would likely result in more money spent, as would viewing more items and spending more time on the website.

The sample proportion or distribution of all metrics was calculated from the user segmentation data set and compared with the sample data of the other segments. For the true proportions, the two-sample proportions z-test was conducted six times to compare every possible duo of segments. This was possible due to the large sample sizes of each cluster and the assumed independence of behaviors between segments. When testing distributions, the Mann-Whitney U Test was used and once again compared for the six possible duos. The Mann-Whitney U Test was used because it does not assume the population distributions are normal, which was the case.

An alpha value of 0.05 was used, meaning the null hypothesis of the proportion or distribution of a segment being

the same as another would have to be less than 5% likely in order to reject.

H. Customer Journey

The customer journey visualizations showed how users interact with different products on the website. It served to find important insights that made users more likely to buy products. Additionally, visualizations were created to conceptualize patterns in items that were added to the cart but not bought.

The abandoned cart analysis first utilized the original data set. The number of items bought is used to determine the purchase rate. Different factors were then analyzed to show whether they had an impact on purchase rate. However, due to instances of the same item being bought hundreds of times from one cart, the inability to verify whether items were bought after they were carted, and the knowledge that most items bought were carted immediately before, a second part of the abandoned cart analysis was completed using session data rather than the original data set.

The session features a data set that was used to see if items carted were bought in the same session. Duration of session, unique products viewed, total number of views, and hour and day of the week carted were all factored to determine what makes a user buy or abandon items in their cart.

I. Price Timing

The price-timing component was built on a `product_price_history` table containing one row per (product, date), with each row recording the median daily price and a 30-day rolling summary statistics(`price_30d_avg`, `price_30d_min`, `price_30d_max`, `price_30d_pct`). Additional features were engineered: lag price and absolute price changes at 1, 3, and 7 days, a 7 day percent price change, 30-day price range and normalized volatility, distance from the 30-day average in absolute and percentage terms, and 3 temporal features: day of the week, month, and day of the month.

The prediction target was binary: will this product's price drop by 5% in the next 7 days? For each observation, the minimum price over the following 7 days using a reverse rolling window, and then we applied a binary threshold. This yielded a heavy imbalance target with 7% positive (price will drop) and 93% negative. Rows with NaN values introduced by lag feature computation were dropped before modeling.

The dataset was split: the earliest 80% of dates for the training set and the latest 20% for testing. This preserves the natural time ordering to prevent look-ahead data leakage. A `HistGradientBoostingClassifier` was trained with `max_iter=200`, `max_depth=4`, `learning_rate=0.05`, and `class_weight='balanced'`. This model was chosen for its native support for missing values and mixed number types common in sparse price data history. Balanced weight was applied to address the 13:1 class imbalance, which prevents the model from defaulting to the majority class. Performance was evaluated against the majority class dummy baseline using ROC-AUC, average precision, precision, and recall. Permutation importance was computed on a 10,000-row test sample to rank the features.

The model's predicted drop probability was made into 3-tiers: Buy Now (<0.3), Monitor (0.3-0.5), and Wait (>=0.5). Recommendations were generated from each product's most

recent observation and stored as a `product_id` to a recommendation, `drop_probability` table.

IV. RESULTS AND DISCUSSION

A. Behavior Analysis

The goal of this notebook was to understand how users interact with the store by analyzing event distributions, conversion behavior, temporal activity patterns, and category/brand popularity through visualizations. Important findings were:

Views account for 94–96% of all events across both months, with cart additions and purchases making up only 2–4% and 1–4% respectively. This imbalance is consistent with normal e-commerce patterns and may provide context for analyzing later results.

Electronics dominated both views and purchases by a significant margin, with Samsung leading at the brand level. Platform-wide popularity is heavily concentrated in a small number of categories and brands.

Temporal analysis revealed that November's daily event volume was consistently higher than October's, with notable spikes around 11–15 and Black Friday. These seasonal differences highlight the importance of accounting for the time period when interpreting any model results or A/B test outcomes.

The hourly activity heatmap identified a consistent peak in user activity between 14:00–18:00 UTC across all days of the week. This finding directly motivates the time-sensitive recommendation system in Notebook 08

B. User Segmentation

TABLE I. USER CLUSTERS

Cluster	Count	Cluster Statistics (Median)				
		views	carts	purchases	total spend	active days
Passive Browsers	3.29M	2	0	0	0	1
One Time Buyers	170K	4	1	1	283.10	1
Active Browsers	1.36M	23	0	0	0	4
Engaged Buyers	495K	42	3	1	272.92	6

K-means clustering was applied across all 5.3 million users, resulting in four distinctive behavioral segments. Categorical features, 'preferred_category' and 'preferred_brand' were not encoded because their high cardinality would create a high-dimensional feature space which could distort distance calculations. The prevalence of certain categories discovered in user behavior analysis also led to the assumption that preferred categories would not provide any meaningful separation.

The optimal k was selected by evaluating inertia and silhouette scores. The elbow method showed the steepest inertia decrease between k=2 and k=4, after which improvement flattened significantly. k=2 was rejected as too broad, and k=4 was selected as it provided a competitive silhouette score with more detailed segments than k=3.

The four resulting segments were profiled by their median feature values and labeled accordingly. Passive Browsers showed low engagement and high recency, indicating infrequent and lapsed users. One-Time Buyers had minimal

views and approximately one purchase with non-zero spend, suggesting users who bought once and did not return. Active Browsers showed high view counts and high cart abandonment but few purchases, capturing users who engage frequently but rarely convert. Engaged Buyers had high views, non-zero purchases, and low recency, representing the store's most valuable recurring users.

Post-clustering analysis confirmed that preferred category distribution was similar across all four segments, validating its exclusion from the clustering features. A PCA reduction to two components was used to visualize the clusters. These components captured ~77% of the original variance and showed reasonable visual separation between clusters, with slight overlap.

K-prototypes is another clustering algorithm that takes both numerical and categorical inputs. This would be a valid or even better alternative to k-means if the categorical user behavior features provided meaningful clustering information.

C. Collaborative Filtering

TABLE II. COLLABORATIVE FILTERING RECALL@5 RESULTS

	<i>Recall@5</i>	<i>Random Baseline</i>
Overall	33.50%	43.45%
Excluding electronics and sparse categories	62.39%	43.45%

The overall Recall@5 of 33.50% falls below the random baseline of 43.45%, explained by two known limitations of cosine similarity in this setting

The first is popularity bias. Electronics was the held-out category for 46.83% of users, yet cosine similarity assigned it near-zero scores with all other categories. Because almost every user interacted with electronics, it does not appear more frequently alongside any one category than another, so cosine similarity was unable to identify it as genuinely similar to any specific category. As a result, electronics was correctly recommended in only 0.83% of cases where it was the held-out item. The second limitation is sparse categories. Medicine, country_yard, and stationery have too few shared users with other categories to produce reliable similarity scores.

When electronics and the three sparse categories are excluded from evaluation, Recall@5 rises to 62.39%, compared to an adjusted random baseline of 43.45%. This suggests the model performs well for the nine categories where sufficient interaction data exists.

A known limitation of the evaluation design is that the leave-one-out evaluation neglects the global timeline, which can lead to data leakage and oversimplification of user preference modeling [9].

D. Time-Sensitive Recommendations

TABLE III. TIME-SENSITIVE RECOMMENDATION RESULTS

	<i>NDCG@5</i>
Baseline (CF only)	0.2117
Temporal (CF + re-ranking)	0.2120

Recall@5 was identical for both models at 33.50%, which is expected since it only measures whether the held-out category appears in the top 5, not where it appears, which is why NDCG@5 was used. NDCG@5 improved from 0.2117 to 0.2120, a relative improvement of 0.13%, which is practically negligible. The average rank change was -0.048, effectively 0. The near-symmetry between cases where the held-out category moved up and down in rank indicates that temporal re-ranking provides no consistent improvement under this evaluation design.

This small improvement is largely a limitation of the evaluation design rather than the re-ranking approach itself. The leave-one-out evaluation holds out each user's highest affinity category, which is not necessarily the category that benefits most from temporal boosting.

Two additional limitations affect the reliability of the temporal signals. Category-level purchase timing was approximated by joining user purchase history with session times through user_id, introducing imprecision in the lift factors. Also, each user's most recent session was used as their temporal context, which assumes users shop at consistent times. This assumption does not always hold in practice.

E. A/B Testing

After conducting Mann-Whitney testing, each cluster's view count distribution, time spent browsing distribution, and money spent distribution were all shown to be statistically different from the others. The p-values were all less than 0.05, indicating each distribution was different from the others. This comes as no surprise due to the large size of each sample. The means and medians for the distributions are provided in Table IV.

TABLE IV. USER SEGMENTATION CLUSTERS

Cluster	Cluster Statistics		
	<i>Median Time (s) Spent Browsing</i>	<i>Median Views</i>	<i>Mean Money (\$) Spent</i>
Engaged Buyers	2819	42	798.79
One-Time Buyers	358	4	633.50
Active Browsers	1289	23	0.71
Passive Browsers	23	2	0.16

After conducting two-proportion Z testing, each cluster's cart and purchase rate was all shown to be statistically different from the others. This comes as no surprise due to the large size of each sample. The p-values were all 0, indicating a statistically significant difference between each of their respective proportions. The sample proportions for the distributions are provided in Table V.

TABLE V. USER SEGMENTATION CLUSTER (CONT.)

Cluster	Sample Proportions	
	<i>Cart Rate</i>	<i>Purchase Rate</i>
Engaged Buyers	0.063	0.035
One-Time Buyers	0.305	0.320
Active Browsers	0.186	0.000
Passive Browsers	0.013	0.001

F. Customer Journey Results

The customer journey analysis revealed that out of all views, 88% went ignored. The remaining 12% of views resulted in a cart of some sort. Of the carts, around 48% of them went on to be bought.

An abandoned cart analysis was needed to look into the remaining 52% of carted items that were not bought. The category with the highest purchase rate once carted was smartphones, and the categories with the lowest purchase rate were apparel, specifically shoes, scarves, and skirts. The analysis also revealed that the purchase rate is highest in the middle of the day and dips during non-working hours from 6 PM to 2 AM. It was also revealed that higher-priced items had higher purchase rates. The \$100-\$500 and \$500+ price bins had the highest purchase rates at 0.48.

At the session level, abandoned carts had lower browsing times, a higher amount of interactions with unique items, and a higher number of items viewed. While sessions where items were bought were largely consistent through the week, sessions with abandoned carts were much higher on Fridays and Saturdays.

G. Price Timing

The HistGradientBoostingClassifier achieved a ROC-AUC of 0.744 on the test set, compared to the 0.5 from the majority-class dummy baseline. Average precision was 0.254, against the baseline of 0.0810.

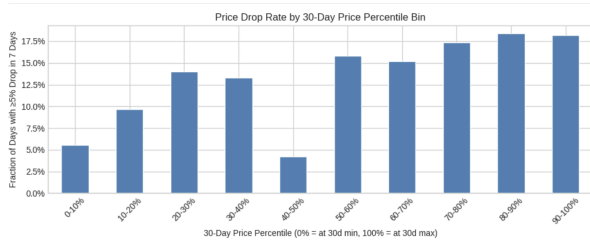


Fig. 1. Price drop rate by 30-day price percentile bin. Products priced at their maximum (90-100th percentile) show the highest likelihood of dropping within the next 7 days.

Exploratory analysis confirmed a strong relationship between the product's position in its 30-day price range and its likelihood of dropping. As shown in Fig. 1, products at their 30-day maximum had the highest drop rate, which motivated price_30d_pct as one of the primary features. The table below presents the full classification report for the gradient boosting classifier at the default threshold of 0.5.

TABLE VI. CLASSIFICATION REPORT

	Classification Report — Default Threshold (0.5)			
	Precision	Recall	F1-Score	Support
No Drop	0.96	0.81	0.88	642,940
Price Drop	0.21	0.58	0.31	56,695
Accuracy	—	—	0.79	699,635
Macro Avg	0.58	0.69	0.59	699,635
Weighted Avg	0.90	0.79	0.83	699,635

At the default threshold, the model achieved a recall of 0.58 with a precision of 0.21. The low precision is acceptable

for this use case because a false Wait recommendation could cost the user a brief delay, while a missed drop can represent a lost savings opportunity.

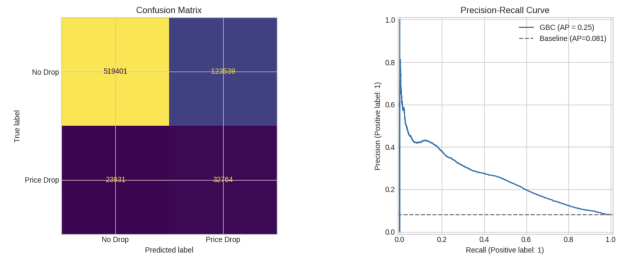


Fig. 2. Confusion matrix (left) and the Precision Recall Curve(right). The PR model achieves an average precision of 0.254, against the baseline of 0.081.

Fig. 2 shows the confusion matrix and the PR curve. At the threshold 0.3, recall is increased to 0.832 with 59.1% of the test set flagged as predicted drops. Fig 3 shows the permutation importance of each feature.

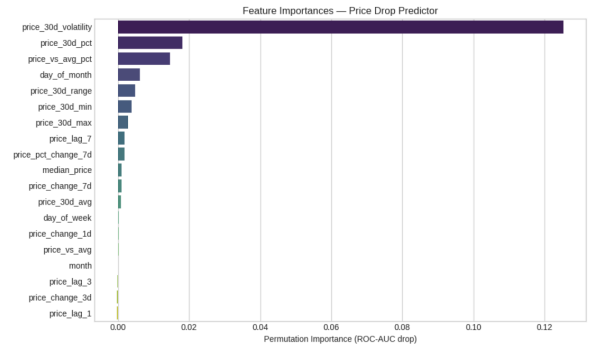


Fig. 3. Permutation feature importance ranked by drop in ROC-AUC.

The exploratory analysis confirmed that the most important feature was price_30d_volatility with a permutation importance of 0.125, higher than all the others. Price_30d_pct (0.018) and price_vs_avg_pct (0.015) ranked second and third. Day_of_month (0.006) ranked highest out of all the temporal features (day of the week, month, and day of the month). Lag and momentum features had minimal importance. Recommendations were generated for about 150,000 products, having 43% Buy Now, 37% Monitor, and 19% Wait.

V. CONCLUSION AND FUTURE WORK

A. Conclusion

This project provided hands-on experience applying core data science methods to a real-world dataset without a predefined project structure or guidance. Working on this project surfaced challenges rarely encountered in structured coursework, from memory management to evaluation design.

B. Future Work

This project was based on a two-month window. Expanding our preprocessing and analysis to include a wider timeframe would help stabilize behavioral patterns and seasonal effects. Expanding our timeframe would allow us to expand from static to time-windowed features and to perform periodic re-clustering, as behavior changes over time.

TF-IDF weighting should be explored to address popularity bias in cosine similarity, which would deflate universally

popular categories and produce more personalized category similarity scores. A live A/B test comparing baseline and temporal recommendations in production would provide a more meaningful evaluation of the temporal re-ranking component.

For the price-timing predictor, a price-timing output directly with the collaborative filtering recommendations would be helpful to produce a single purchase signal.

REFERENCES

- [1] M. Kechinov, "eCommerce behavior data from multi-category store," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store>
- [2] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, Jan. 2003. doi: 10.1109/MIC.2003.1167344
- [3] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 2008 IEEE Int. Conf. Data Mining (ICDM '08)*, Washington, DC, USA, 2008, pp. 263–272. doi: 10.1109/ICDM.2008.22
- [4] Y. Koren, "Collaborative filtering with temporal dynamics," in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '09)*, Paris, France, 2009, pp. 447–456. doi: 10.1145/1557019.1557072
- [5] I. Etzioni, R. Tsuchida, C. A. Knoblock, and A. Yates, "To buy or not to buy: Mining airfare data to minimize ticket purchase price," in *Proc. 9th ACM SIGKDD*, 2003, pp. 119–128.
- [6] Song & Liu — "An XGBoost Algorithm for Predicting Purchasing Behaviour on E-Commerce Platforms," *Tehnički vjesnik*, Vol. 27, No. 5, pp. 1467–1471, 2020. DOI: 10.17559/TV-20200808113807
- [7] Liu et al. — "Machine learning-based e-commerce platform repurchase customer prediction model," *PLOS ONE*, Dec. 2020. DOI: 10.1371/journal.pone.0243105
- [8] Pedregosa et al. — "Scikit-learn: Machine learning in Python," *JMLR*, Vol. 12, pp. 2825–2830, 2011.
- [9] R. Maystre and M. Grossglauser, "Just Enough Data: Avoiding Overfitting in Recommender Systems," *arXiv:2010.11060*, 2020. [Online]. Available: <https://arxiv.org/abs/2010.11060>
- [10] M. Alves Gomes and T. Meisen, "A review on customer segmentation methods for personalized customer targeting in e-commerce use cases," *Information Systems and e-Business Management*, vol. 21, no. 3, pp. 527–570, 2023, doi: 10.1007/s10257-023-00640-4.